RunAndSchedule2Survive: Algorithm Scheduling Based on Run2Survive

VALENTIN MARGRAF, MCML, LMU Munich, Munich, Germany
TOM KOERNER, LMU Munich, Munich, Germany
ALEXANDER TORNEDE, Beckhoff Automation GmbH und Co KG, Verl, Germany
MARCEL WEVER, L3S Research Center, Leibniz University Hannover, Hannover, Germany

The algorithm selection problem aims to identify the most suitable algorithm for a given problem instance under specific time constraints, where suitability typically refers to a performance metric such as algorithm runtime. While previous work has employed machine learning techniques to tackle this challenge, methods from survival analysis have proven particularly effective. This article presents RunAndSchedule2Survive to address the more general and complex problem of algorithm scheduling, where the objective is to allocate computational resources across multiple algorithms to maximize performance within specified time constraints. Our approach combines survival analysis with evolutionary algorithms to optimize algorithm schedules by leveraging runtime distributions modeled as survival functions. Experimental results across various standard benchmarks demonstrate that our approach significantly outperforms previous methods for algorithm scheduling and yields more robust results than its algorithm selection variant. More specifically, RunAndSchedule2Survive achieves superior performance in 20 out of 25 benchmark scenarios, surpassing hitherto state-of-the-art approaches.

CCS Concepts: • **Computing methodologies** → **Supervised learning**; **Search methodologies**; **Planning under uncertainty**;

Additional Key Words and Phrases: algorithm scheduling, algorithm selection, survival analysis

ACM Reference format:
Valentin Margraf, Tom Koerner, Alexander Tornede, and Marcel Wever. 2025. RunAndSchedule2Survive: Algorithm Scheduling Based on Run2Survive. *ACM Trans. Evol. Learn. Optim.* 5, 3, Article 21 (August 2025), 17 pages.
https://doi.org/10.1145/3737705

Marcel Wever acknowledges funding by the European Union (ERC, ´´ixAutoML", grant no. 101041029). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council Executive Agency. Neither the European Union nor the granting authority can be held responsible for them. The authors gratefully acknowledge the computing time provided to them on the high-performance computers Noctua2 at the NHR Center PC2. These are funded by the Federal Ministry of Education and Research and the state governments participating on the basis of the resolutions of the GWK for the national highperformance computing at universities (www.nhr-verein.de/unsere-partner).

Authors' Contact Information: Valentin Margraf (corresponding author), MCML, LMU Munich, Munich, Germany; e-mail: valentin.margraf@ifi.lmu.de; Tom Koerner, LMU Munich, Munich, Germany; e-mail: t.koerner@campus.lmu.de; Alexander Tornede, Beckhoff Automation GmbH und Co KG, Verl, Germany; e-mail: a.tornede@ai.uni-hannover.de; Marcel Wever, L3S Research Center, Leibniz University Hannover, Hannover, Germany; e-mail: m.wever@ai.uni-hannover.de.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 2688-3007/2025/8-ART21
https://doi.org/10.1145/3737705

1 Introduction

The performance of algorithms for computationally complex problems, such as Boolean satisfiability [Cook, 1971], Traveling salesman [Dantzig et al., 1954], or **Constraint Satisfaction Problems (CSPs)**, can vary greatly depending on the specific problem instance at hand [Gomes et al., 1997]. In particular, no single algorithm consistently outperforms all others across all instances of a given problem, a phenomenon known as *performance complementarity* [Gomes and Selman, 2001]. The algorithm selection problem [Rice, 1976] addresses this challenge by attempting to predict which algorithm will perform best for a specific problem instance within a given time budget (i.e., has the shortest runtime), typically leveraging methods from machine learning [Kerschke et al., 2019; Kotthoff, 2014].

Previous work introduced a novel approach to algorithm selection called Run2Survive [Tornede et al., 2020], which leverages survival analysis techniques [Kleinbaum and Klein, 2005], a branch of statistics traditionally used for time-to-event data. By modeling algorithm runtime behavior as survival distributions, the risk of deteriorated performance can be assessed and weighted according to a given risk-aversion level. While redefining the state of the art in algorithm selection, the predictions of this approach can also be imperfect and may be subject to uncertainty [Tornede et al., 2023], potentially resulting in inferior performance.

A common technique to increase the robustness of algorithm selection approaches is to schedule multiple algorithms and allocate the computational resources among them, instead of dedicating the entire runtime budget to a single algorithm [Lindauer et al., 2016]. In this work, we extend Run2Survive and integrate the survival analysis models with an evolutionary algorithm that optimizes algorithm schedules with respect to expected runtime as anticipated from predicted survival curves. To this end, we devise a method of combining independent survival curves in a principled way that serves as a foundation for evolutionary optimization.

Contributions. The key contributions are threefold:

- We propose a method for combining survival curves of different algorithms into a joint survival curve representing the survival curve of an algorithm schedule.
- We complement the framework Run2Survive [Tornede et al., 2020] by an evolutionary algorithm that optimizes algorithm schedules based on the predicted survival curves for an unseen problem instance. With this, we propose the first algorithm scheduling method that computes algorithm schedules by taking into account the anticipated instance-specific runtime behavior of every algorithm of a schedule.
- We validate our approach on a comprehensive set of benchmark problems from the commonly used benchmarking library ASLib [Bischl et al., 2016] and demonstrate its superiority over existing algorithm scheduling [Amadini et al., 2014; Hoos et al., 2012; Liu et al., 2021; Malitsky et al., 2012; Streeter et al., 2007] as well as algorithm selection methods [Amadini et al., 2014; Kadioglu et al., 2010; Tornede et al., 2020; Xu et al., 2008].

The remainder of this article is organized as follows: Section 2 introduces the algorithm selection and algorithm scheduling problems while surveying existing works. Section 3 gives a brief introduction to survival analysis before we motivate algorithm scheduling with the help of survival functions in Section 4. In Sections 5 and 6, we propose and validate our proposed method RunAndSchedule2Survive. Finally, we conclude and outline directions of future work in Section 7.

2 Algorithm Selection and Scheduling

Algorithm Selection [Kerschke et al., 2019; Kotthoff, 2014] and *Algorithm Scheduling* [Lindauer et al., 2016] are strategies designed to solve problem instances efficiently within a specified time budget.

In algorithm selection, a single algorithm is chosen to run for the entire time budget. In contrast, algorithm scheduling involves selecting one or multiple suitable algorithms and distributing the available runtime among them. We will provide more thorough introduction to algorithm selection and algorithm scheduling in Sections 2.1 and 2.2, respectively.

2.1 Algorithm Selection

Algorithm selection aims at identifying the most suitable algorithm from a (fixed) portfolio of algorithms to execute on a given problem instance within a certain time budget. This selection is typically based on historical performance data of the algorithm and feature characteristics of the given problem instance [Kotthoff, 2014]. In combinatorial optimization, some algorithms take extremely long to solve certain problem instances [Gomes et al., 1997]. To save computational cost, the time budget is typically limited by an upper bound κ. Algorithms with runtimes exceeding κ are forcefully terminated at κ and penalized, often according to the well-known $PAR10$ score, which assigns a performance value of $10 \cdot \kappa$ when termination occurs.

Formally, per-instance algorithm selection [Rice, 1976] deals with a problem instance space $\mathcal{I}$, a set of algorithms $\mathcal{A}$, and a performance measure $m : \mathcal{I} \times \mathcal{A} \to \mathbb{R}$, typically measuring algorithm runtime. The objective is to determine for each instance $i \in \mathcal{I}$ a mapping, called algorithm selector, $\gamma : \mathcal{I} \to \mathcal{A}$ from that instance to an algorithm $a \in \mathcal{A}$ with minimal expected runtime. The optimal algorithm selector is called oracle or **Virtual Best Solver (VBS)** and is defined as:

$$\gamma^*(i) \in \arg\min_{a \in \mathcal{A}} \mathbb{E}[m(i, a)].$$

In the literature, various approaches to algorithm selection have been proposed. For instance, the first framework for automated SAT solver selection, SATzilla [Xu et al., 2008], is trained on diverse features extracted from SAT instances and observed solver performance data. During inference, SATzilla predicts the best-performing SAT solver on a per-instance basis. AutoFolio [Lindauer et al., 2015] employs an algorithm configuration procedure to address the performance variability of different algorithm selection techniques across different problem domains. Specifically, it automatically selects and configures an appropriate algorithm selection approach amongst, e.g., SATzilla, SAT Solver Selector, and others. Run2Survive [Tornede et al., 2020] adopts *Survival Analysis* [Kleinbaum and Klein, 2005] to the algorithm selection setting. Specifically, they employ random survival forests [Ishwaran et al., 2008] to model the runtime distribution of an algorithm for a given problem instance. These predicted runtime distributions enable selecting the most suitable algorithm based on criteria such as the minimal expected runtime or the lowest probability of a timeout.

2.2 Algorithm Scheduling

Algorithm scheduling involves selecting a portfolio of suitable algorithms from the pool of algorithm candidates given a specific problem instance [Lindauer et al., 2016]. Instead of allocating the entire budget κ to a single algorithm, a schedule comprising multiple algorithms is constructed, and the runtime is distributed among them. While the best choice would still be a schedule of length 1 comprising the fastest algorithm, in the face of uncertainty and non-deterministic algorithm performances, algorithm schedules help to increase robustness. Algorithm schedule methods can be either static or dynamic. *Static* methods select multiple algorithms based on training instances and apply them uniformly across all test instances. In contrast, *dynamic* methods compute tailored schedules for each test instance, often leveraging instance-specific features.

Similar to algorithm selection, algorithm scheduling operates on a problem instance space $\mathcal{I}$ and a set of algorithms $\mathcal{A}$. However, the performance measure $m : \mathcal{I} \times \Sigma \to \mathbb{R}$ depends on an entire schedule $\sigma \in \Sigma$. The goal is to find a mapping, called algorithm scheduler, $\lambda : \mathcal{I} \to \Sigma$ from

an instance i to an algorithm schedule $\sigma : \mathbb{N} \rightarrow \mathbb{N} \times [0, \kappa]$. This schedule specifies both the order of the algorithms, denoted by a permutation function $\tau : \mathbb{N} \rightarrow \mathbb{N}$ over the algorithm indices, and the end runtime of each algorithm. The runtime for each algorithm is determined by the difference between its allocated end runtime and that of the previous algorithm in the sequence. The goal is to predict a schedule with minimum expected runtime and the optimal algorithm scheduler (oracle scheduler) is defined as:

$$\lambda^*(i) \in \arg\min_{\sigma \in \Sigma} \mathbb{E}[m(i, \sigma)].$$

An early approach to algorithm scheduling, **Combining Multiple Heuristics Online (CMHO)** [Streeter et al., 2007], computes an approximately optimal schedule for the given training instances per unit time. In the offline setting, this (static) schedule is afterwards used for all test instances, assuming that training and test instances are drawn from the same probability distribution. Aspeed [Hoos et al., 2012] employs **Answer Set Programming (ASP)** [Baral, 2010] to design a predetermined, static algorithm schedule tailored to specific problem instances. The authors formulate the determination of a schedule as multi-criteria optimization problem, to be able to provide corresponding ASP encodings.

Survival analysis has been adapted as well in the context of algorithm scheduling, notably in a bandit-based approach proposed by Gagliolo and Schmidhuber [2006]. This strategy involves simultaneously running all candidate algorithms while utilizing a model to learn their respective runtime distributions, which requires that algorithm executions can be paused and resumed. The following dynamic approaches construct per-instance schedules under the assumption that algorithms performing well on similar instances might also excel for the current instance, where similarities between problem instances are computed through Euclidean distances between their feature vectors. For example, the **SAT Solver Selector (3S)** [Malitsky et al., 2011] selects the algorithm capable of solving most of the k most similar problem instances to the given instance and allocates 90% of the budget to this selected solver. The remaining 10% are devoted to a static schedule. This approach can also be parallelized, as proposed by Malitsky et al. [2012], and is called P3S. SUNNY [Amadini et al., 2014] combines a diverse set of CSP solvers into a schedule by selecting, again, those that perform well on the k most similar (training) problem instances. It then devises a schedule for the algorithms within the portfolio. Meanwhile, Liu et al. [2021] present sunny-as2, an enhanced version of SUNNY. sunny-as2 introduces two refinements: Firstly, the neighborhood parameter k is tuned to the dataset instead of fixing its value *a priori*, as in SUNNY. Additionally, it conducts feature selection by identifying the optimal subset of features. For a more comprehensive overview of algorithm selection and algorithm scheduling, we refer to Kerschke et al. [2019] and Lindauer et al. [2016].

3 Basics of Survival Analysis for Algorithm Selection

We base our algorithm scheduling approach on the algorithm selector Run2Survive [Tornede et al., 2020], which in turn is based on survival analysis to model algorithm runtime distributions. In this section, we revisit the core concepts of survival analysis and random survival forests, adhering to the notation established by Tornede et al. [2020].

3.1 Basic Concepts

Generally speaking, in *Survival Analysis* [Kleinbaum and Klein, 2005], one is interested in the time until a certain event occurs. A dataset is represented as $\mathcal{D} = \{(\vec{x}_i, y_i, \delta_i)\}_{i=1}^{N}$, with $\vec{x}_i \in \mathcal{X} \subseteq \mathbb{R}^d$ being the context, $y_i \in \mathcal{Y} \subseteq \mathbb{R}_+$ the observed time until the event occurs for the given context, and $\delta_i \in \{0, 1\}$ indicating the censoring status of the sample. If the sample is uncensored, i.e.,

$\delta_i = 0$, y_i represents the true time; otherwise, a clipped version thereof. The goal is to learn a model $z : \mathcal{X} \to \mathbb{R}^+$ that predicts the expected survival time $T \in \mathbb{R}^+$ for a given context $\vec{x} \in \mathcal{X}$. The survival time is considered a random variable and represented by a survival function defined as:

$$S(t, \vec{x}) = \mathbb{P}[T \geq t \mid \vec{x}].$$

The survival function models the probability of the event occurring at time t or later, given the context $\vec{x}$. The complementary function, the *cumulative death distribution* function, is denoted as $F(t, \vec{x}) = \mathbb{P}[T \leq t \mid \vec{x}]$ with $S(t, \vec{x}) = 1 - F(t, \vec{x})$. This concept is closely linked to the *hazard function*, defined as:

$$h(t, \vec{x}) = \lim_{\Delta_t \to 0} \frac{\mathbb{P}[t \leq T < t + \Delta_t \mid T \geq t, \vec{x}]}{\Delta_t},$$

which describes the instantaneous probability for the event to occur at timepoint t, provided it has not occurred already. By defining the cumulative hazard function $H(t, \vec{x}) = \int_0^t h(u, \vec{x}) du$ one can derive the survival function via the relation $S(t, \vec{x}) = \exp(-H(t, \vec{x}))$. As already stated, the model should learn to predict the expected runtime, which is the expectation of the random variable T:

$$z(\vec{x}) = \mathbb{E}[T] = \int_0^\infty S(t, \vec{x}) dt = \int_0^\infty 1 - F(t, \vec{x}) dt.$$

3.2 Survival Analysis for Algorithm Selection

As proposed in Tornede et al. [2020], survival analysis can be applied to the setting of algorithm selection, where one is faced with the following datasets:

$$\mathcal{D}_a = \{(i, m(i, a), \delta_{m(i,a)=\kappa}) \mid i \in \mathcal{I}_\mathcal{D}\}.$$

Here, i denotes a specific problem instance from a given set $\mathcal{I}_\mathcal{D}$ to be solved by an algorithm a. The event of interest $m(i, a)$ represents the runtime of algorithm a to solve instance i. If the runtime exceeds the cutoff time κ, and is hence clipped, i.e., $m(i, a) = \kappa$, it constitutes a censored sample.

Given this setting, one is interested in a model z that predicts the survival function of an algorithm given a specific problem instance. Random survival forests [Ishwaran et al., 2008] offer a suitable choice for modeling the survival functions S_a of an algorithm a and are also utilized in Tornede et al. [2020]. These random survival forests estimate survival step functions $\hat{S}_a$ for each algorithm by building multiple random trees. Similar to standard random forests, they leverage bootstrapping, i.e., training each tree on a random subset of the survival time data $\mathcal{D}_a$. At each node of the tree, the algorithm searches for the best split based on a chosen criterion [Breiman et al., 1984]. In survival analysis, this split is determined by maximizing the difference between child nodes [Ishwaran et al., 2008]. During prediction, the random survival forest combines the predictions of its ensemble members by providing a survival probability curve for the time-to-event outcome in the form of the cumulative hazard function. The expected runtime can then be computed by approximating the integral over the step function as follows:

$$\int_0^\infty S_a(t, i) dt \approx \sum_{t_k, t_{k+1} \in \mathcal{T}} t_k \cdot \left[\hat{S}_a(t_k, i) - \hat{S}_a(t_{k+1}, i)\right],$$

where $\mathcal{T}$ denotes the set of event times of the given algorithm. Based on the expected runtime, different criteria can be used to select suitable algorithms, e.g., minimal expected runtime [Schoemaker, 1980] or the lowest timeout probability. Generally speaking, in the context of algorithm selection, survival analysis is applied with the opposite objective of the traditional use cases—the goal is to achieve an early termination rather than a long survival, as this signifies that the algorithm has efficiently found a solution.

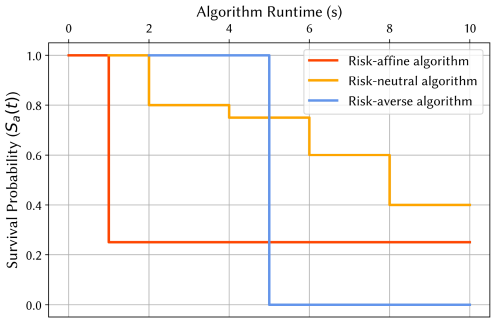

Fig. 1. Survival curves of risk-affine, risk-averse, and risk-neutral algorithms.

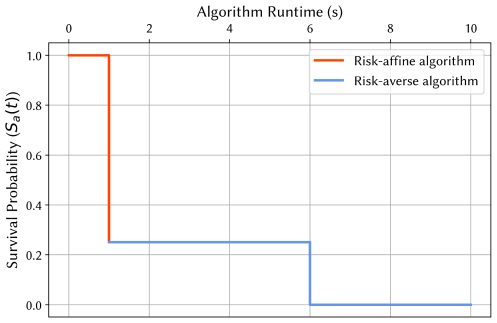

Fig. 2. Survival curve of the schedule constructed from the risk-affine and risk-averse algorithm.

4 Motivation for Scheduling Algorithms Based on Survival Curves

While survival analysis methods have proven beneficial for the setting of algorithm selection, survival curves could also be informative for the related problem of dynamic algorithm scheduling. The process of constructing a schedule becomes particularly valuable when considering a portfolio of algorithms with diverse risk preferences, each exhibiting distinct behaviors within their survival curves. A *risk-averse* algorithm's survival curve typically depicts a low survival probability in the end but covers more area under the curve, resulting in a higher expected runtime. Despite potentially taking longer to solve an instance, this algorithm has a notably high chance of completing within the given time limit and thus avoiding a PAR10 penalty. In contrast, a *risk-affine* algorithm portrays an abrupt drop in its survival curve early on, indicating a higher chance of solving the problem instance quite quickly. However, it does not guarantee a steep decline toward zero probability near the cutoff, leaving a higher risk for a timeout, as depicted in Figure 1.

The advantage of scheduling these different types of algorithms instead of selecting a single one lies in the complementary nature of their strengths. Initially scheduling the risk-affine algorithm aims to quickly process the instance. However, to ensure a high probability of solving the instance within the runtime cutoff, the remaining time budget is then allocated to a more risk-averse algorithm.

Explicitly, consider the example of three algorithms and their corresponding survival curves as shown in Figure 1 and a cutoff time of 10 seconds. The *risk-affine* algorithm drops to a survival probability of 0.25 after 1 second, while the *risk-averse* algorithm drops to 0 after 5 seconds. The survival curve of the *risk-neutral* algorithm gradually decreases but does not fall below 0.4. When constructing an algorithm schedule based on the information offered by the survival curves, we would begin the schedule with the risk-affine algorithm. However, we would only let it run for 1 second as for larger time budgets its survival probability no longer decreases. Subsequently, we would start the risk-averse algorithm and let it run until the cutoff time is reached. Note that, since the risk-averse algorithm starts later, its survival curve in Figure 2 is shifted to the right compared to Figure 1. However, combining the two survival curves yields a survival curve as shown in Figure 2, which exhibits a lower expected runtime than any of the two algorithms.

5 RunAndSchedule2Survive: Evolving Algorithm Schedules Based on Survival Curves

Similar to Run2Survive [Tornede et al., 2020], we train random survival forests in an offline phase to predict per-instance survival curves. Specifically, we train the forests using problem instance features as inputs, with the target being pairs of a binary event indicator and the time of the

Algorithm 1: Construction of a Survival Curve from an Algorithm Schedule

Require: Schedule $\sigma : j \mapsto \tau(j) \times t_j$, Algorithm survival functions $\mathcal{S} = [S_{a_1}, ..., S_{a_n}]$
1: $S_\sigma(0) \leftarrow 1.0$ ▷ *Initialize schedule survival curve.*
2: $t_0 \leftarrow 0$ ▷ *Initialize starting time.*
3: **for** $j = 1 \dots n$ **do**
4: $(\tau(j), t_j) \leftarrow \sigma(j)$ ▷ *Get algorithm index and end time.*
5: $S_\sigma(t_{j-1} + t) \leftarrow \min\{S_\sigma(t_{j-1}), S(t)\} \quad \forall t \in (0, t_j - t_{j-1}]$
6: **end for**
7: **return** S_σ

event. In the online phase, the model predicts survival probabilities for test instances at each event time, which are inferred from the training data where events occurred. From these probabilities the survival curve is constructed. Our proposed method then constructs schedules from these (predicted) survival curves via evolutionary optimization, as is detailed in Sections 5.1 and 5.2. Moreover, to streamline the optimization process by reducing the size of the search space, we introduce a simple yet effective pruning technique to ignore algorithms that cannot be part of an optimal algorithm schedule (Section 5.3).

5.1 Schedule Survival Curve Construction

Starting with the survival curves predicted by Run2Survive, we aim to assess the risk of an algorithm schedule. We hence need a method to combine survival functions of individual algorithms obtained through the models of Run2Survive into a survival function for the algorithm schedule. For a given problem instance $i \in \mathcal{I}$ and a portfolio of algorithms $a \in \mathcal{A}$, we assume to be given an algorithm schedule in the form of a function $\sigma_i(j) := (\tau(j), t_j)$ (as defined in Section 2.2). Here, $\tau(\cdot)$ denotes a permutation on the indices of the algorithms' indices $j \in \{1, \dots, |\mathcal{A}|\}$. While $\tau(j)$ yields the index of the algorithm, $t_j \in [0, \kappa]$ refers to the end time within the global time window, ranging from 0 to the cutoff time κ. Moreover, we assume to be given the survival functions of the individual algorithms $S_{a_\tau(j)}(t, i)$ for a problem instance i at time t. For the sake of readability, we omit the specific instance i, on which the survival curves of all algorithms depend in the subsequent notation, and simply write $S_{a_{\tau(j)}}(t)$ instead. Similarly, we write $\sigma(j)$ instead of $\sigma_i(j)$.

Based on the survival functions of the individual algorithms, we construct the survival function S_σ of an algorithm schedule σ consisting of n algorithms inductively as follows. We depart from an empty schedule with a survival probability of 1 at time 0, i.e., $S_\sigma(0) = 1$, and initialize with $t_0 = 0$. Iterating over the algorithms scheduled at positions $j = 1, \dots, n$ with $\sigma(j) = (\tau(j), t_j)$, we set:

$$S_\sigma(t) = \min\{S_\sigma(t_{j-1}), S_{a_{\tau(j)}}(t - t_{j-1})\} \qquad \forall t \in \left(t_{j-1}, t_j\right].$$

For the last algorithm in the algorithm schedule, we set the end time to κ, indicating that this algorithm runs until the cutoff time. We do this regardless of its actual value since in the event of not solving the problem instance, the entire schedule will typically be punished with a penalty score anyway, e.g., $10 \cdot \kappa$. The iterative construction of the algorithm schedule's survival curve is also presented in Algorithm 1.

Note that when switching to the next algorithm of a schedule, we retain the lowest survival probability of the preceding algorithm as long as the newly scheduled algorithm does not undercut it. While the survival probability of the preceding algorithm might have decreased after early terminating it, this is not reflected in the survival curve of the schedule since the algorithm is no longer executed. Furthermore, as we cannot transfer partial solutions between algorithms, the next algorithm starts from scratch, which means that the jth algorithm is given a timeslice of size $t_j - t_{j-1}$ starting from time 0 of the new algorithm.

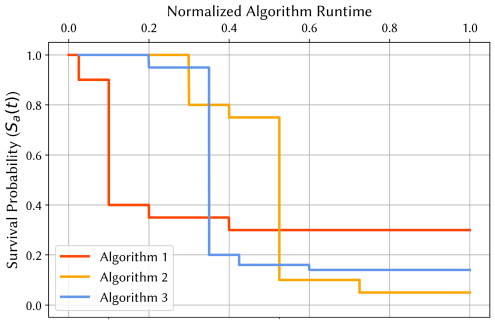

Fig. 3. Survival curves of three different algorithms. Runtimes are normalized, i.e., $\kappa = 1$.

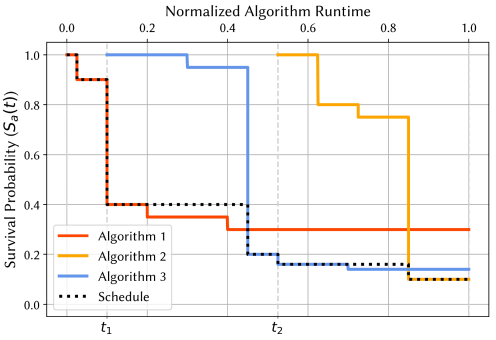

Fig. 4. Construction of the survival curve for the schedule $\sigma = [(1, 0.1 \cdot \kappa), (3, 0.525 \cdot \kappa), (2, \kappa)]$ from the individual survival curves of the algorithms.

For example, consider the survival functions in Figures 3 and the survival function of an algorithm schedule constructed as described above in Figure 4. We can see that for a schedule $\sigma = [(1, 0.1 \cdot \kappa), (3, 0.525 \cdot \kappa), (2, \kappa)]$, Algorithm 1 is scheduled first and terminated early at $0.1 \cdot \kappa$. Although the survival probability of Algorithm 1 decreases further in the timeslice of $0.1 \cdot \kappa$ to $0.5 \cdot \kappa$, the survival curve of the schedule remains at a survival probability of 0.4 since Algorithm 1 has already been terminated and Algorithm 3 has not yet undercut the survival probability of Algorithm 1. Only at $0.5 \cdot \kappa$ the survival probability of Algorithm 3 drops below 0.4, affecting the survival probability of the schedule. The same behavior can be observed for executing Algorithm 2 after Algorithm 3 is terminated early at $0.525 \cdot \kappa$.

5.2 Evolving Algorithm Schedules from Survival Curves

Now, that we can construct survival functions for algorithm schedules from survival functions from individual algorithms as predicted by Run2Survive's models, we can assess the quality of a specific algorithm schedule. As described in Section 2.2, based on a survival function, we can calculate the expected runtime for an algorithm schedule or weighted versions thereof. While this facilitates assessing the expected quality of an algorithm schedule, optimizing the actual algorithm schedule is a non-trivial problem on its own. Therefore, we employ a genetic algorithm to optimize the permutation of algorithms and the assigned size of the timeslice.

For evolving the order and the timeslice of the algorithms within an algorithm schedule, it is sufficient to evolve the end times of every algorithm, as by the order of the end times also an order on the algorithms is implied, resulting in a genetic representation of $[0, \kappa]$. However, since the cutoff times κ can be sometimes relatively large, genetic operators would need to be configured for every scenario. Therefore, we normalize the time interval and only operate on the $[0, 1]$ interval. Additionally, algorithms can only be effectively excluded from the schedule if their runtime is set to 0, which is unlikely to happen in an evolutionary process. Therefore, to facilitate excluding algorithms from a schedule, we add a Boolean variable to the genetic representation to denote for every algorithm whether it is included in the schedule or not. All in all, for an algorithm portfolio $\mathcal{A}$, we obtain a genetic representation of the form:

$$(\{0, 1\} \times [0, 1])^{|\mathcal{A}|}.$$

This genetic representation is amenable to standard genetic algorithms. While for this work, we solely aim for a proof-of-concept, since the genetic algorithm needs to be executed for every

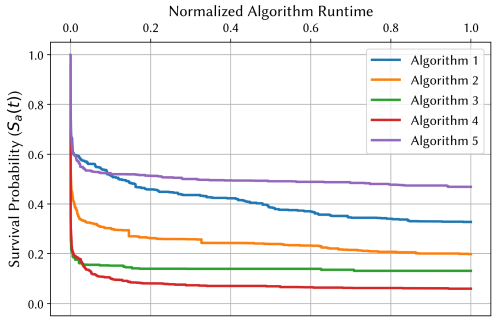

Fig. 5. Predicted survival curves of different algorithms on problem instance 1 of the scenario QBF-2011 from ASlib.

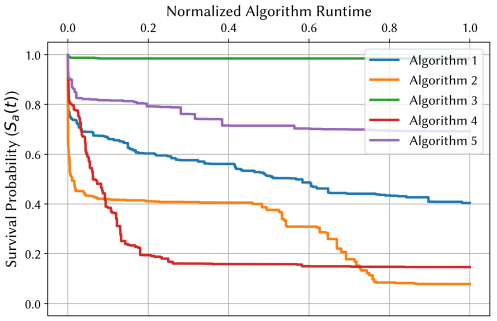

Fig. 6. Predicted survival curves of different algorithms on problem instance 6 of the scenario QBF-2011 from ASlib.

problem instance, a genetic algorithm customized for this particular problem could be beneficial to reduce computational costs for optimizing an algorithm schedule. However, we leave this for future work and employ a standard genetic algorithm in our evaluation.

Eventually, to complete the setup of the genetic algorithm, we need a fitness function to assess the quality of the algorithm schedules represented by individuals of the population. For this, we obtain the schedule σ by ordering the algorithms according to the evolved end times, yielding $\tau(j)$ and the end times directly represent $t_{\tau(j)}$. Algorithms with the Boolean gene set to 0 are not considered in the schedule, by reducing the algorithm portfolio for that particular problem instance to the remaining algorithms. The fitness of an individual can then be assessed by calculating the expected runtime or weighted versions thereof.

5.3 Preprocessing and Initialization

While the general setup of the genetic algorithm appears to be relatively simple, in scenarios with larger portfolios, the genetic algorithm struggles to find optima due to the higher dimensionality of the optimization problem. Fortunately, in real-world scenarios, we can observe two patterns in the predicted survival curves that allow us to improve the efficiency of our approach. Specifically, we often encounter situations as shown in Figures 5 and 6, which show predicted survival curves of different algorithms for the ASlib scenario QBF-2011 for two different problem instances.

In Figure 5, we can observe that the single survival function of Algorithm 4 dominates every other survival function. Meaning that at any time the survival probability is lower than for any other survival curve. Consequently, we can conclude that the optimal algorithm schedule is of length 1 and consists of Algorithm 4 because it results in the lowest expected runtime. In this case, we do not optimize for an algorithm schedule at all and, instead, return an algorithm schedule of length 1 comprising only the dominating algorithm and let it run for the full budget.

Alternatively, we can observe a situation as shown in Figure 6. Here, we can make two observations. First, there is not a single survival function dominating all other survival functions. Hence, optimizing for an algorithm schedule beyond a single algorithm is beneficial. However, second, we can observe some survival functions that are dominated by a set of other survival curves, i.e., they never achieve the lowest survival probability at any point in time. In the figure, the dominated algorithms are Algorithms 1, 3, and 5. Consequently, these algorithms would never be included in an optimal algorithm schedule. While we could leave these decisions to the optimization process, pruning these algorithms prior to evolving algorithm schedules helps to reduce the search space

size and thus increase search efficiency. Moreover, pruning those algorithms can be done safely without excluding the optimum.

Taking together the two observations, we propose to employ a preprocessing step to prune the search space prior to running the genetic algorithm. If the survival function of one algorithm dominates all other algorithms, we even skip the execution of the genetic algorithm completely, as already described before.

Last but not least, by randomly initializing the genetic algorithm, it is still difficult to reach smaller algorithm schedules that oftentimes perform superior to longer schedules. Therefore, we seed the initial population with individuals representing schedules of length 1, one individual per algorithm, with randomly sampled end times. While in the fitness evaluation, the end time of the algorithm is set to κ anyway, sampling random end times randomly can help recombine algorithm schedules of length 1 into slightly longer algorithm schedules. The remaining population is initialized completely at random.

6 Empirical Evaluation

In this section, we evaluate the proposed approach RunAndSchedule2Survive, which we configure as detailed in Section 6.1, on a standard collection of algorithm selection benchmark ASlib as detailed in Section 6.2. In the experiments, we compare RunAndSchedule2Survive to a diverse set of algorithm selection and scheduling baselines outlined in Section 6.3, including the extended state-of-the-art algorithm selector Run2Survive. The experiment setup is detailed in Section 6.4, before we present and discuss the results of our experiments in Section 6.5.

6.1 Configuration of RunAndSchedule2Survive

For the implementation of RunAndSchedule2Survive, we employ a standard genetic algorithm framework.[1] We ensure that the initial population contains at least one individual for each algorithm, and then randomly sample additional individuals to reach a population size of 100. The optimization runs for a maximum number of 10,000 iterations. To foster exploration, we perform soft restarts every 10 generations, keeping only the top 5 elitist individuals and replacing the remaining population with randomly sampled individuals. Furthermore, we employ an early termination criterion and terminate the genetic algorithm if the best fitness cannot be improved for 25 generations. Single-point crossover and mutation are applied with probabilities of 0.9 and 0.1, respectively. As described in Section 5.3, we seed the initial population with individuals representing algorithm schedules of length 1. For every algorithm of the algorithm portfolio, we create one such individual and sample individually completely at random until we fill up the population.

6.2 Datasets

The *Algorithm Selection Library* ASlib [Bischl et al., 2016] contains a collection of datasets of a wide variety of different scenarios along with the performance of different algorithms on these datasets. Precisely, we evaluate the baselines and our approach on the ASlib scenarios described in Table 1.

6.3 Baselines

We compare our approach against both algorithm selection and algorithm scheduling methods. More specifically, we consider five algorithm schedulers, outlined in Section 6.3.1, and three algorithm selectors (Section 6.3.2). In the rare case where an approach crashed and thus failed to solve an instance, like, e.g., Aspeed on GRAPHS-2015, we impute with the performance of the **Single Best Solver (SBS)**.

[1] https://github.com/PasaOpasen/geneticalgorithm2.

Table 1. ASlib Scenarios on Which We Evaluated the Baselines and Our Approach

	ASP-POTASSCO	BNSL-2016	CPMP-2015	CSP-2010	CSP-Minizinc-Time-2016	GRAPHS-2015	MAXSAT-PMS-2016	MAXSAT-WPMS-2016	MAXSAT12-PMS	MAXSAT15-PMS-INDU	MIP-2016	PROTEUS-2014	QBF-2011	QBF-2014	QBF-2016	SAT03-16_INDU	SAT11-HAND	SAT11-INDU	SAT11-RAND	SAT12-ALL	SAT12-HAND	SAT12-INDU	SAT12-RAND	SAT15-INDU	TSP-LION2015
$\lvert\mathcal{I}\rvert$	1,294	1,179	527	2,024	100	5,725	601	630	876	601	218	4,021	1,368	1,254	825	2,000	296	300	600	1,614	767	1,167	1,362	300	3,106
$\lvert\mathcal{A}\rvert$	11	8	4	2	20	7	19	18	6	29	5	22	5	14	24	10	15	18	9	31	31	31	31	28	4
#feat	138	86	22	86	95	35	37	37	37	37	143	198	46	46	46	483	115	115	115	115	115	115	115	54	122
κ	600	7,200	3,600	5,000	1,200	10^8	1,800	1,800	2,100	1,800	7,200	3,600	3,600	900	1,800	5,000	5,000	5,000	5,000	1,200	1,200	1,200	1,200	3,600	3,600

The number of instances, the number of algorithms, and the number of features are represented by $|\mathcal{I}|$, $|\mathcal{A}|$, and #feat, respectively. The runtime cutoff is given by κ.

6.3.1 *Algorithm Schedulers.*

CMHO [Streeter et al., 2007]. This approach computes an approximately optimal (static) schedule for the given training instances in a greedy manner, such that the number of instances solved per unit time is maximized. Afterwards, this schedule is applied to the test instances, assuming that training and test instances are drawn from the same probability distribution.

Aspeed [Hoos et al., 2012]. The algorithm scheduling framework Aspeed leverages ASP for optimizing schedules for given test problem instances. This framework applies a single (static) schedule across all test instances, which was obtained by minimizing timeouts based on the training instances.

P3S [Malitsky et al., 2012]. In this scheduling approach, a portfolio of algorithms is selected based on their performance on similar training instances. The scheduling involves allocating 90% of the budget to the algorithm that solved the most k-nearest instances in the training set, where k is tuned by an internal cross-validation procedure. The remaining budget is allocated to a schedule of all available algorithms. This schedule is computed via integer programming, strategically allocating time slots for each algorithm to maximize the number of overall solved training instances. Our implementation does not involve parallelization. However, to reduce computational costs without significantly affecting performance, we implemented a few adjustments: We set a 180-second timeout for computing the fixed schedule and utilized a 3-fold Monte Carlo cross-validation for internal hyperparameter tuning of k. Further, k was chosen from a set of reasonable neighborhood sizes, namely 2, 5, 10, and 20.

SUNNY [Amadini et al., 2014]. A minimal portfolio of algorithms is selected that have efficiently solved the most k-nearest instances in the shortest time, with k being set to 16. The time slots are computed proportionally, considering each algorithm's number of successful instances within the k-nearest instances. To reduce computational costs, we capped the maximum size of the portfolio at 5, as SUNNY typically selected a compact portfolio comprising no more than two or three solvers in the majority of cases.

sunny-as2 [Liu et al., 2021]. This approach extends on SUNNY by introducing additional enhancements. The hyper-parameter k is finetuned and feature selection is conducted in an initial offline phase. Subsequently, SUNNY is run using the optimized parameter value for k and the chosen feature subset. Various versions of sunny-as2 are proposed: *sunny-as2-k*, focusing solely on tuning k; *sunny-as2-f*, dedicated only to feature selection; and *sunny-as2-fk*, involving both aspects. For our implementation, we opted for sunny-as2-k due to its strong performance and significantly reduced

computational costs. We fine-tuned k by selecting from the set consisting of 1, 5, 10, 15, 20, and 30 using a greedy variant of SUNNY, as outlined in Liu et al. [2021].

6.3.2 *Algorithm Selectors.*

PerAlgorithmRegressor. This algorithm selector trains a separate Random Forest for each candidate algorithm to predict its performance. For a given instance, the algorithm with the best-estimated performance is selected.

Instance-Specific Algorithm Configuration (ISAC) [Kadioglu et al., 2010]. In ISAC, similar problem instances are grouped in the form of clusters and annotated with the algorithm that performs best on average for that cluster. In the application phase, new problem instances are matched with the closest cluster centroid and the annotated algorithm is returned as a prediction.

SATzilla'11 [Xu et al., 2008]. SATzilla'11 reduces the algorithm selection problem to pairwise comparisons between algorithms. A Decision Forest is trained for each pair to determine which algorithms perform better for a given problem instance. For any new instance, all Decision Forests cast a vote, and the algorithm with the most votes from the pairwise comparisons is selected. Ties are broken by choosing the algorithm with the lower average runtime on the training data.

SUNNY-like. Similar to Amadini et al. [2014], we evaluate each algorithm based on the number of instances it successfully solves within the k-nearest range, with k set to 16. However, only the algorithm that resolves the highest number of instances is chosen.

Run2Survive [Tornede et al., 2020]. Leveraging random survival forests to model the runtime distribution of an algorithm, Run2Survive effectively estimates the expected runtime by integrating over the runtime distribution. For a given instance the algorithm with minimal expected runtime is chosen. Among the three variants available, we chose the one that utilizes polynomial and log-based surrogate loss functions as this variant demonstrated the strongest performance. In this approach, both the type of surrogate loss and its associated parameters are optimized through Bayesian optimization using the training data.

6.4 Experimental Setup

We evaluate all approaches on the 10 predefined train–test splits for each scenario of the ASlib benchmark collection. Further, we impute missing values by the mean of the corresponding feature and only consider test instances for which at least one algorithm could solve the instance, i.e., has a runtime below the fixed timeout. We also do not apply any pre-solver or feature selector in our evaluation.

To compare the performances of the approaches, we compute the average normalized *penalized average runtime* (nPAR10) scores over the 10 folds for each problem scenario. The PAR10 score represents the average time required to solve a specific problem instance. In cases where the runtime of an algorithm surpasses the predefined cutoff time, a runtime value of 10 times the cutoff time is assigned. Based on observed PAR10 scores, the normalized PAR10 score is defined as $\text{nPAR10} := \text{PAR10}_{model} - \text{PAR10}_{VBS}/\text{PAR10}_{SBS} - \text{PAR10}_{VBS}$. Hereby, PAR10_{SBS} and PAR10_{VBS} define the PAR10 score of SBS and VBS, respectively.

We conducted all experiments on a single machine equipped with AMD EPYC MILAN 7413 Processor with 24 cores at 2.65 GHz and 256 GB RAM. We provide an implementation of our approach.[2]

[2] https://github.com/ValentinMargraf/run_and_schedule_to_survive/.

Table 2. Normalized PAR10 Scores of Evaluated *Algorithm Schedule Approaches*, Where a Score of 1 Indicates a Performance Equal to the SBS

Scenario name	CMHO	Aspeed	P3S	SUNNY	sunny-as2	**Ours**
ASP-POTASSCO	0.96 ± 0.10	3.42 ± 2.38	1.24 ± 0.48	0.89 ± 0.50	0.83 ± 0.24	**0.22 ± 0.10**
BNSL-2016	0.95 ± 0.14	0.45 ± 0.29	0.42 ± 0.17	0.55 ± 0.16	0.95 ± 0.28	**0.18 ± 0.14**
CPMP-2015	0.64 ± 0.25	0.76 ± 0.25	0.65 ± 0.33	0.79 ± 0.50	**0.50 ± 0.19**	0.69 ± 0.31
CSP-2010	0.87 ± 0.24	13.6 ± 16.4	0.54 ± 0.34	2.65 ± 2.95	5.14 ± 2.87	**0.18 ± 0.24**
CSP-Minizinc-Time-2016	0.72 ± 0.46	5.71 ± 1.56	0.66 ± 0.66	0.92 ± 1.01	0.53 ± 0.52	**0.36 ± 0.46**
GRAPHS-2015	0.99 ± 0.10	1.00 ± 0.00	**0.51 ± 0.65**	4.71 ± 11.1	2.70 ± 5.91	2.26 ± 5.79
MAXSAT-PMS-2016	1.10 ± 0.47	5.26 ± 5.23	0.89 ± 0.54	1.09 ± 0.81	0.94 ± 0.96	**0.49 ± 0.44**
MAXSAT-WPMS-2016	0.92 ± 0.40	11.8 ± 9.65	2.93 ± 2.00	1.93 ± 1.63	1.45 ± 1.20	**0.36 ± 0.27**
MAXSAT12-PMS	0.91 ± 0.28	8.02 ± 2.04	0.36 ± 0.29	0.73 ± 0.38	0.75 ± 0.39	**0.13 ± 0.14**
MAXSAT15-PMS-INDU	1.06 ± 0.50	15.4 ± 11.4	0.83 ± 0.45	0.97 ± 0.57	1.01 ± 0.64	**0.35 ± 0.31**
MIP-2016	3.44 ± 7.15	**0.68 ± 0.26**	8.24 ± 23.2	3.14 ± 7.17	0.84 ± 0.42	5.28 ± 11.4
PROTEUS-2014	0.60 ± 0.11	2.40 ± 1.15	0.29 ± 0.07	0.57 ± 0.31	1.38 ± 0.08	**0.29 ± 0.05**
QBF-2011	0.91 ± 0.14	3.05 ± 1.10	0.40 ± 0.12	0.97 ± 0.24	0.49 ± 0.19	**0.11 ± 0.07**
QBF-2014	1.00 ± 0.00	1.07 ± 1.20	0.82 ± 0.22	1.1 ± 0.33	0.67 ± 0.17	**0.25 ± 0.13**
QBF-2016	**0.04 ± 0.01**	4.96 ± 2.16	0.66 ± 0.23	1.20 ± 0.43	0.63 ± 0.22	0.32 ± 0.11
SAT03-16_INDU	1.01 ± 0.11	4.27 ± 1.64	1.01 ± 0.33	0.87 ± 0.21	1.02 ± 0.25	**0.58 ± 0.12**
SAT11-HAND	1.03 ± 0.86	2.78 ± 1.89	0.51 ± 0.37	0.61 ± 0.35	0.63 ± 0.37	**0.32 ± 0.24**
SAT11-INDU	1.03 ± 0.45	1.54 ± 1.37	0.93 ± 0.33	1.03 ± 0.34	0.91 ± 0.46	**0.58 ± 0.39**
SAT11-RAND	0.31 ± 0.12	2.30 ± 0.82	0.13 ± 0.09	0.49 ± 0.29	0.38 ± 0.17	**0.06 ± 0.05**
SAT12-ALL	0.97 ± 0.04	2.55 ± 0.78	0.69 ± 0.13	0.86 ± 0.16	0.87 ± 0.13	**0.28 ± 0.04**
SAT12-HAND	1.03 ± 0.04	2.19 ± 0.43	0.49 ± 0.17	0.55 ± 0.26	0.65 ± 0.18	**0.26 ± 0.08**
SAT12-INDU	1.06 ± 0.15	3.51 ± 0.65	1.39 ± 0.55	0.95 ± 0.23	1.12 ± 0.39	**0.50 ± 0.34**
SAT12-RAND	1.00 ± 0.00	17.1 ± 9.55	1.29 ± 0.45	1.10 ± 0.44	2.81 ± 1.85	**0.80 ± 0.29**
SAT15-INDU	0.94 ± 0.37	9.99 ± 3.41	1.07 ± 0.49	1.22 ± 0.44	1.35 ± 0.48	**0.79 ± 0.58**
TSP-LION2015	**0.69 ± 0.40**	0.77 ± 0.27	7.35 ± 6.55	40.7 ± 77.3	30.7 ± 23.7	1.66 ± 0.34
Average score	0.97	4.98	1.37	2.82	2.37	**0.69**
Mean rank	3.54	5.32	2.96	3.98	3.66	**1.54**

Lower is better and bold indicates the best result for each scenario. The standard deviation of the normalized PAR10 scores across the 10 train–test splits is denoted by ±.

6.5 Results

In the following, we present the results of our approach compared to the baseline algorithm schedulers in Table 2 and algorithm selectors in Table 4. The comparisons comprise the 25 ASlib scenarios listed in Table 1, reporting the nPAR10 scores. A value of 1 corresponds to the performance of the SBS, whereas a value of 0 means VBS performance. Values between 0 and 1, therefore, indicate to what extent the gap between SBS and VBS can be closed by the corresponding methods, and a lower score means a better performance.

As shown in Table 2, our approach consistently outperforms the baseline schedulers across most scenarios, with the exceptions being CPMP-2015, GRAPHS-2015, QBF-2016, and TSP-LION2015. In these scenarios, the algorithm selector Run2Survive also does not perform too well (see Table 4), indicating that the predicted survival functions may not accurately approximate the actual survival functions of the algorithms. We hypothesize that the likely poor-quality predictions of the survival

Table 3. Detailed Results of Our Approach across Different Scenarios, Including the Initial Number of Algorithms ($|\mathcal{A}|$), the Average Number of Algorithms after Pre-Filtering ($|\mathcal{A}_{\text{pre}}|$), the Final Optimized Schedule Length ($|\sigma_{\text{opt}}|$), and the Average Time for Optimizing a Schedule in Seconds (t_{opt})

	ASP-POTASSCO	BNSL-2016	CPMP-2015	CSP-2010	CSP-Minizinc-Time-2016	GRAPHS-2015	MAXSAT-PMS-2016	MAXSAT-WPMS-2016	MAXSAT12-PMS	MAXSAT15-PMS-INDU	MIP-2016	PROTEUS-2014	QBF-2011	QBF-2014	QBF-2016	SAT03-16_INDU	SAT11-HAND	SAT11-INDU	SAT11-RAND	SAT12-ALL	SAT12-HAND	SAT12-INDU	SAT12-RAND	SAT15-INDU	TSP-LION2015
$\lvert\mathcal{A}\rvert$	11	8	4	2	20	7	19	18	6	29	5	22	5	14	24	10	15	18	9	31	31	31	31	28	4
$\lvert\mathcal{A}_{\text{pre}}\rvert$	7.01	4.79	2.53	1.72	5.40	3.77	5.92	4.86	2.67	7.04	2.01	4.49	2.62	4.75	4.91	5.57	5.19	9.70	5.35	5.65	5.13	8.43	4.06	10.24	3.69
$\lvert\sigma_{\text{opt}}\rvert$	1.39	1.59	1.10	1.17	1.56	1.47	1.63	1.84	1.23	1.58	1.07	1.45	1.24	1.27	2.11	1.20	1.33	1.01	1.48	1.22	1.15	1.16	1.42	1.23	1.67
t_{opt}	2.77	2.80	1.39	1.53	2.15	3.35	3.09	2.93	1.48	3.79	0.98	4.30	1.26	2.06	3.41	2.75	1.69	1.83	1.86	3.83	2.24	3.20	2.73	3.13	5.33

Table 4. Normalized PAR10 Scores of Evaluated *Algorithm Selection Approaches (and Our Approach)*, Where a Score of 1 Indicates a Performance Equal to the SBS

Scenario name	PerAlgoReg	ISAC	SATzilla'11	SUNNY-like	Run2Survive	**Ours**
ASP-POTASSCO	0.32 ± 0.09	0.65 ± 0.21	0.43 ± 0.15	**0.17 ± 0.08**	0.32 ± 0.14	0.22 ± 0.10
BNSL-2016	0.20 ± 0.15	0.83 ± 0.09	0.20 ± 0.10	0.25 ± 0.13	0.2 ± 0.10	**0.18 ± 0.14**
CPMP-2015	0.88 ± 0.48	0.98 ± 0.46	0.85 ± 0.29	1.05 ± 0.44	0.91 ± 0.69	**0.69 ± 0.31**
CSP-2010	0.37 ± 0.46	0.37 ± 0.33	0.24 ± 0.35	0.40 ± 0.44	0.24 ± 0.41	**0.18 ± 0.24**
CSP-Minizinc-Time-2016	0.51 ± 0.47	0.70 ± 0.42	0.50 ± 0.52	0.52 ± 0.69	0.46 ± 0.49	**0.36 ± 0.46**
GRAPHS-2015	5.96 ± 16.4	4.64 ± 11.8	8.23 ± 23.0	11.5 ± 34.6	2.42 ± 5.35	**2.26 ± 5.79**
MAXSAT-PMS-2016	0.75 ± 0.35	1.02 ± 0.71	0.59 ± 0.42	0.49 ± 0.47	0.58 ± 0.34	**0.49 ± 0.44**
MAXSAT-WPMS-2016	0.47 ± 0.25	0.76 ± 0.15	0.43 ± 0.38	**0.24 ± 0.29**	0.39 ± 0.27	0.36 ± 0.27
MAXSAT12-PMS	0.30 ± 0.26	0.49 ± 0.21	0.24 ± 0.17	0.28 ± 0.13	0.29 ± 0.21	**0.13 ± 0.14**
MAXSAT15-PMS-INDU	0.54 ± 0.36	1.01 ± 0.35	0.43 ± 0.28	**0.24 ± 0.21**	0.42 ± 0.31	0.35 ± 0.31
MIP-2016	10.1 ± 23.4	3.03 ± 7.35	9.66 ± 23.1	8.86 ± 23.5	**0.81 ± 0.49**	5.28 ± 11.4
PROTEUS-2014	0.45 ± 0.03	0.64 ± 0.05	0.58 ± 0.06	0.47 ± 0.06	0.41 ± 0.05	**0.29 ± 0.05**
QBF-2011	0.20 ± 0.09	0.33 ± 0.10	0.17 ± 0.09	0.22 ± 0.10	0.20 ± 0.09	**0.11 ± 0.07**
QBF-2014	0.33 ± 0.18	0.52 ± 0.17	0.36 ± 0.17	0.40 ± 0.26	0.31 ± 0.17	**0.25 ± 0.13**
QBF-2016	0.63 ± 0.28	0.58 ± 0.19	0.64 ± 0.26	0.51 ± 0.23	0.49 ± 0.2	**0.32 ± 0.11**
SAT03-16_INDU	0.85 ± 0.33	0.96 ± 0.19	0.88 ± 0.28	0.85 ± 0.30	0.73 ± 0.14	**0.58 ± 0.12**
SAT11-HAND	0.45 ± 0.26	0.63 ± 0.29	0.45 ± 0.22	0.70 ± 0.41	0.47 ± 0.45	**0.32 ± 0.24**
SAT11-INDU	0.62 ± 0.29	0.95 ± 0.52	0.84 ± 0.44	0.85 ± 0.56	0.71 ± 0.51	**0.58 ± 0.39**
SAT11-RAND	0.11 ± 0.07	0.58 ± 0.11	0.37 ± 0.14	0.13 ± 0.08	0.12 ± 0.09	**0.06 ± 0.05**
SAT12-ALL	0.36 ± 0.11	0.68 ± 0.08	0.43 ± 0.08	0.40 ± 0.10	0.37 ± 0.08	**0.28 ± 0.04**
SAT12-HAND	0.36 ± 0.11	0.63 ± 0.10	0.31 ± 0.05	0.42 ± 0.15	0.33 ± 0.06	**0.26 ± 0.08**
SAT12-INDU	0.82 ± 0.28	0.99 ± 0.25	0.64 ± 0.22	0.81 ± 0.32	0.75 ± 0.24	**0.50 ± 0.34**
SAT12-RAND	1.24 ± 0.47	1.09 ± 0.29	0.92 ± 0.19	1.11 ± 0.39	0.95 ± 0.33	**0.80 ± 0.29**
SAT15-INDU	0.97 ± 0.44	**0.75 ± 0.46**	0.80 ± 0.35	1.04 ± 0.35	1.00 ± 0.65	0.79 ± 0.58
TSP-LION2015	2.16 ± 1.58	25.0 ± 20.1	41.3 ± 29.3	11.0 ± 8.26	2.56 ± 2.86	**1.66 ± 0.34**
Average score	1.20	1.95	2.82	1.72	**0.66**	0.69
Mean rank	3.82	5.18	3.72	4.12	2.90	**1.26**

Lower is better and bold indicates the best result for each scenario. The standard deviation of the normalized PAR10 scores across the 10 train–test splits is denoted by ±.

Table 5. Win Matrix: Each Entry (i, j) Represents the Number of Scenarios in Which the Approach in Row i Significantly Outperforms the Approach in Column j Based on the Wilcoxon Signed-Rank Test with $p < 0.05$

	CMHO	Aspeed	P3S	SUNNY	sunny-as2	PerAlgoReg	ISAC	SATzilla'11	SUNNY-like	Run2Survive	**Ours**
CMHO	0	1	5	4	5	2	4	3	3	2	2
Aspeed	1	0	1	1	2	1	2	1	2	1	1
P3S	9	18	0	11	9	2	6	5	2	1	0
SUNNY	3	18	3	0	4	0	1	0	1	0	0
sunny-as2	5	18	4	4	0	1	2	1	1	1	0
PerAlgoReg	16	19	7	14	15	0	12	4	2	0	0
ISAC	7	18	5	7	6	0	0	1	0	0	0
SATzilla'11	14	19	12	15	14	2	12	0	4	0	0
SUNNY-like	14	19	9	12	14	4	14	6	0	1	0
Run2Survive	17	19	13	15	16	2	15	4	4	0	0[a]
Ours	17	19	15	21	20	12	18	11	12	10	0

[a]Considering the difference in mean performance for MIP-2016 a win of Run2Survive over our method would be expected. However, there are only a few outliers distorting the mean of our method, resulting in the null hypothesis not being rejected. Still, the p-value for this setting, being at 0.13, is relatively close to the threshold of 0.05.

functions hinder both the selection of a single algorithm and the informed construction of an algorithm schedule. In the MIP-2016 scenario, our approach also performs poorly, despite Run2Survive achieving strong results. This suggests that factors beyond survival function quality may influence scheduling performance, needing further investigation.

The overall effectiveness of our approach is further illustrated by the mean rank score, where our method achieves a rank of 1.54, and by the average nPAR10 score of 0.69, which is clearly superior to the baselines. Moreover, our method demonstrates remarkable robustness across the evaluated scenarios, consistently outperforming the SBS in nearly all cases. This underlines the practical applicability and reliability of our method in real-world scenarios.

Comparing our approach to the algorithm selectors, we outperform all baselines on 20 ASlib scenarios. Run2Survive achieves better results on MIP-2016, ISAC on SAT15-INDU, and SUNNY on ASP-POTASSCO, MAXSAT-WPMS-2016, and MAXSAT-15-PMS-INDU, see Table 4. While our method ranks highest, with an average rank of 1.26, we do not surpass Run2Survive in terms of the average nPAR10 score. This discrepancy is primarily due to the outlier in the MIP-2016 scenario, where we recorded a high nPAR10 score of 5.28 for our approach. Nevertheless, these results underscore the advantage of constructing robust algorithm schedules rather than relying on the selection of only a single algorithm.

In Table 3, we present detailed metrics on our approach's performance for each problem scenario. Precisely, we report: (1) the initial set of algorithms available before pre-filtering, (2) the average number of algorithms that remain after the pre-filtering step, (3) the optimized schedule length after running our method, and (4) the mean time required for optimizing the schedules. As can be seen, the pre-filtering step often significantly reduces the set of algorithms, frequently cutting the initial pool by half or even more. After optimization, the final schedule length is consistently between one and two algorithms across all scenarios, indicating that only a few algorithms are typically needed to achieve high performance. Lastly, the mean time taken for optimization remains consistently low (under 6 seconds on a single CPU core), likely a result of the initial filtering step, which allows the optimization to proceed more quickly over a reduced candidate set. For further reducing the optimization time, parallelization and more sophisticated optimization methods, e.g., evolutionary strategies, could be used.

Lastly, in Table 5, we present the results of statistical tests comparing each approach against every other using the Wilcoxon signed-rank test. The results highlight that Run2Survive is a strong competitor, as it rarely loses to others. However, our approach outperforms certain methods more frequently, such as PerAlgoReg (12 wins vs. 2 losses), as well as SATzilla'11 and SUNNY-like. Notably, in a direct comparison, we win against Run2Survive 10 times and never lose. Also for the MIP-2016 scenario Run2Survive does not show a statistically significant advantage ($p = 0.13$), despite having a significantly lower normalized PAR10 score. This is due to the mean performance being skewed by two strong outliers in the runs for our approach. Overall, these results further confirm that our method holds up under statistical assessment.

7 Conclusion and Future Work

In this article, we proposed to leverage survival analysis for the problem of dynamic algorithm scheduling. Extending Run2Survive [Tornede et al., 2020], we predict runtime distributions for algorithms, which we utilize to construct schedules customized for specific problem instances. Comparing our method to Run2Survive and four other algorithm selection methods, as well as five different algorithm scheduling approaches on a variety of 25 benchmark scenarios from the ASlib collection, our findings highlight the robustness and superior performance of our approach. Explicitly, we outperform the baseline schedulers as well as the baseline selectors on 20 out of 25 scenarios.

Regarding future work, we aim to improve our proposed approach through the following strategies. Firstly, a straightforward yet potentially impactful enhancement involves utilizing a more advanced optimizer for the genetic optimization process. Our initial implementation focused on simplicity and uses a standard general-purpose genetic algorithm, but employing a more sophisticated optimizer holds promise, particularly considering that the majority of our approach's runtime involves creating and assessing each schedule with the fitness function. Additionally, instead of relying only on the PAR10 score for schedule evaluation, we aim to explore the use of a specific polynomial function. This function would further prioritize schedules featuring risk-affine algorithms in the initial stages and transitioning towards risk-averse ones later on. This direction aligns with suggestions from Tornede et al. [2020] for their algorithm selection approach. Lastly, an area of improvement for our method involves preemptive scheduling, which allows for pausing and resuming algorithms at a later point while maintaining their exact state. This feature could be beneficial, as illustrated in the scenario depicted in Figure 6. In the past, this has also been proposed and implemented for other scheduling approaches [Gagliolo and Schmidhuber, 2006; Kadioglu et al., 2011]. However, this extension also increases the complexity of the schedule construction problem, so it would be necessary to investigate to what extent the increased effort is reflected in terms of performance gains.

References

Roberto Amadini, Maurizio Gabbrielli, and Jacopo Mauro. 2014. SUNNY: A lazy portfolio approach for constraint solving. *Theory and Practice of Logic Programming* 14, 4–5 (2014), 509–524.

Chitta Baral. 2010. *Knowledge Representation, Reasoning and Declarative Problem Solving*. Cambridge University Press, Cambridge.

Bernd Bischl, Pascal Kerschke, Lars Kotthoff, Marius Lindauer, Yuri Malitsky, Alexandre Fréchette, Holger H. Hoos, Frank Hutter, Kevin Leyton-Brown, Kevin Tierney, et al. 2016. ASlib: A benchmark library for algorithm selection. *Artificial Intelligence* 237 (2016), 41–58.

Leo Breiman, J. H. Friedman, Richard A. Olshen, and C. J. Stone. 1984. *Classification and Regression Trees*. Wadsworth, New York, USA.

Stephen A. Cook. 1971. The complexity of theorem-proving procedures. In *3rd Annual ACM Symposium on Theory of Computing*. ACM, 151–158.

George B. Dantzig, D. Ray Fulkerson, and Selmer M. Johnson. 1954. Solution of a large-scale traveling-salesman problem. *Journal of the Operations Research Society of America* 2, 4 (1954), 393–410.

Matteo Gagliolo and Jürgen Schmidhuber. 2006. Learning dynamic algorithm portfolios. *Annals of Mathematics and Artificial Intelligence* 47, 3–4 (2006), 295–328.

Carla P. Gomes and Bart Selman. 2001. Algorithm portfolios. *Artificial Intelligence* 126, 1–2 (2001), 43–62.

Carla P. Gomes, Bart Selman, and Nuno Crato. 1997. Heavy-tailed distributions in combinatorial search. In *Principles and Practice of Constraint Programming (CP '97)*. Lecture Notes in Computer Science, Vol. 1330, Springer, 121–135.

Holger H. Hoos, Roland Kaminski, Torsten Schaub, and Marius Schneider. 2012. Aspeed: ASP-based solver scheduling. In *Technical Communications of the 28th International Conference on Logic Programming (ICLP). LIPIcs*, Vol. 17, Schloss Dagstuhl—Leibniz-Zentrum für Informatik, 176–187.

Hemant Ishwaran, Udaya B. Kogalur, Eugene H. Blackstone, and Michael S. Lauer. 2008. Random survival forests. *The Annals of Applied Statistics* 2, 3 (2008), 841–860.

Serdar Kadioglu, Yuri Malitsky, Ashish Sabharwal, Horst Samulowitz, and Meinolf Sellmann. 2011. Algorithm selection and scheduling. In *Principles and Practice of Constraint Programming (CP). Lecture Notes in Computer Science*, Vol. 6876, Springer, 454–469.

Serdar Kadioglu, Yuri Malitsky, Meinolf Sellmann, and Tierney Kevin. 2010. ISAC—Instance-specific algorithm configuration. In *19th European Conference on Artificial Intelligence (ECAI '10). Frontiers in Artificial Intelligence and Applications*, Vol. 215, IOS Press, 751–756.

Pascal Kerschke, Holger H. Hoos, Frank Neumann, and Heike Trautmann. 2019. Automated algorithm selection: Survey and perspectives. *Evolutionary Computation* 27, 1 (2019), 3–45.

David G. Kleinbaum and Mitchel Klein. 2005. *Survival Analysis: A Self-Learning Text*. Springer Science and Business Media, LLC.

Lars Kotthoff. 2014. Algorithm selection for combinatorial search problems: A survey. *AI Magazine* 35, 3 (2014), 48–60.

Marius Lindauer, Rolf-David Bergdoll, and Frank Hutter. 2016. An empirical study of per-instance algorithm scheduling. In *10th International Conference on Learning and Intelligent Optimization (LION). Lecture Notes in Computer Science*, Vol. 10079, Springer, 253–259.

Marius Lindauer, Holger H. Hoos, Frank Hutter, and Torsten Schaub. 2015. AutoFolio: An automatically configured algorithm selector. *Journal of Artificial Intelligence Research* 53 (2015), 745–778.

Tong Liu, Roberto Amadini, Maurizio Gabbrielli, and Jacopo Mauro. 2021. sunny-as2: Enhancing SUNNY for algorithm selection. *Journal of Artificial Intelligence Research* 72 (2021), 329–376.

Yuri Malitsky, Ashish Sabharwal, Horst Samulowitz, and Meinolf Sellmann. 2011. Non-model-based algorithm portfolios for SAT. In *Theory and Applications of Satisfiability Testing (SAT '11)*. Springer, Berlin, 369–370.

Yuri Malitsky, Ashish Sabharwal, Horst Samulowitz, and Meinolf Sellmann. 2012. Parallel SAT solver selection and scheduling. In *18th International Conference on Principles and Practice of Constraint Programming (CP). Lecture Notes in Computer Science*, Vol. 7514, Springer, 512–526.

John R. Rice. 1976. The algorithm selection problem. *Advances in Computers* 15 (1976), 65–118.

Paul J. H. Schoemaker. 1980. *Expected Utility Theory*. Springer, Dordrecht, 11–27.

Matthew J. Streeter, Daniel Golovin, and Stephen F. Smith. 2007. Combining multiple heuristics online. In *22nd AAAI Conference on Artificial Intelligence*. AAAI Press, 1197–1203.

Alexander Tornede, Lukas Gehring, Tanja Tornede, Marcel Wever, and Eyke Hüllermeier. 2023. Algorithm selection on a meta level. *Machine Learning* 112, 4 (2023), 1253–1286.

Alexander Tornede, Marcel Wever, Stefan Werner, Felix Mohr, and Eyke Hüllermeier. 2020. Run2Survive: A decision-theoretic approach to algorithm selection based on survival analysis. In *12th Asian Conference on Machine Learning (ACML)*. Proceedings of Machine Learning Research, Vol. 129, PMLR, 737–752.

Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. 2008. SATzilla: Portfolio-based algorithm selection for SAT. *Journal of Artificial Intelligence Research* 32 (2008), 565–606.

Received 12 January 2024; revised 21 March 2025; accepted 11 May 2025